20251349.pdf

Vellore Institute of Technology - Andhra Pradesh

Document Details

Submission ID
trn:oid:::3618:136011250

Submission Date
Apr 21, 2026, 4:20 PM GMT+5:30

Download Date
Apr 21, 2026, 5:09 PM GMT+5:30

File Name
20251349.pdf

File Size
219.5 KB

6 Pages

4,411 Words

24,031 Characters

2% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text
- Small Matches (less than 8 words)

Match Groups

6 Not Cited or Quoted 2%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
1% Publications
2% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **6** Not Cited or Quoted 2%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
1% Publications
2% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	
Monash University on 2025-09-05		<1%
2	Internet	
arxiv.org		<1%
3	Publication	
Yamparala Srinivas, Lanka Swathi, Tippani Deepika, Kodali Samuel Sanjay, Kilar...		<1%
4	Internet	
ca-larkspur.civicplus.com		<1%
5	Submitted works	
Instituto de Empresa S.L.U. on 2026-04-10		<1%
6	Internet	
theses.hal.science		<1%

StockSense AI: An Intelligent Inventory Management and Supplier Decision Support System for Small Retail Stores

Vyshakh Vijayan

***Abstract*—India has over 12 million kirana stores, and most of them still track inventory by hand. Expensive enterprise software is out of reach for these shop owners, so they end up guessing when to reorder stock. StockSense AI addresses this gap. It is a web-based inventory intelligence system that runs on mostly free-tier cloud services and costs under Rs. 500 a month in production. The architecture has four layers: a React 19 frontend, Supabase PostgreSQL for storage, n8n for workflow automation, and Groq Llama 3.1 8B for generating reorder suggestions. Every morning at 8:00 AM IST, a master workflow runs eight sub-pipelines that calculate stockout dates, classify risk levels, score suppliers with a Weighted Sum Model, produce AI-driven reorder recommendations, and push a summary to the store owner's WhatsApp via Twilio. Testing with six products and three suppliers shows that the full pipeline finishes in roughly 13 seconds, alerts arrive within 5 seconds, and the health-score formula behaves as expected across extreme inventory states.**

***Index Terms*—Inventory Management, Supply Chain Intelligence, Workflow Automation, n8n, Large Language Model, Supplier Scoring, Kirana Stores, SME Technology**

I. Introduction

WALK into any kirana store in India and you will notice the same thing—the owner tracks stock in a notebook or, at best, a basic spreadsheet. With over 12 million such neighborhood shops in the country, this manual approach leads to two recurring problems. Either the shelves run empty and customers leave, or money sits locked in unsold inventory. Both outcomes eat into already thin margins.

Why does this persist? For one, enterprise software such as Zoho Inventory starts at Rs. 4,000 a month, which is impractical for a shop earning Rs. 15,000–20,000 in monthly profit. Simpler POS apps like Khatabook and Vyapar do exist, but they only record past sales—they cannot predict what will run out next week or suggest how much to reorder. On top of that, Indian supply chains are fragmented: a single store may deal with five to ten suppliers, each with different reliability and delivery speeds, making vendor selection a daily guessing game.

StockSense AI was built to fill this exact gap. It is an end-to-end inventory intelligence system for small Indian retailers, and it runs on mostly free-tier cloud services. The five goals driving the project are: (1) track stock in real time and predict upcoming stockouts, (2) generate reorder suggestions using a large language model, (3) rank suppliers through a Weighted Sum Model, (4) send WhatsApp alerts automatically, and (5) keep the total cost under Rs. 500 per month.

What makes this work different from a typical inventory app is the automation layer. Every morning at 8:00 AM, a master orchestration workflow triggers eight sub-pipelines that update stockout dates, flag risky products, call the Groq Llama 3.1 8B model for reorder reasoning, score suppliers, and finally push a WhatsApp digest to the shop owner. The store owner does not need to open the dashboard at all—the system comes to them.

The rest of this paper is laid out as follows. Section II reviews related work. Section III explains the motivation and problem specification. Section IV describes the architecture and mathematical models. Section V covers implementation details. Section VI discusses results, and Section VII concludes with future scope.

II. Related Work

Several lines of research inform the design of StockSense AI. This section groups them into four themes: smart inventory systems, demand forecasting, LLM-based decision support, and workflow automation.

On the inventory systems front, Wang et al. [1] built an intelligent warehousing system for small enterprises and reported clear gains in stock accuracy. Their paper also noted, however, that adoption among micro-retailers stays low because even "affordable" solutions assume internet literacy and upfront hardware. This observation matches what we see in Indian kirana stores, where the owner is often the sole employee.

Forecasting is a well-studied area. Kheawpeam and Sinthupinyo [2] showed that ML models beat traditional moving-average methods by 15–25% for multi-channel retailers. Soni and Vaghela [3] extended this by adding external signals like festivals and promotions. Both studies, though, target organized retail chains with historical POS data spanning months. A kirana shop that has never digitized its sales does not have that data, which is why StockSense currently relies on a simpler rule-based stockout formula rather than trained ML models.

Large language models are beginning to appear in business decision-making. Al Khaldy and Gheraibia [4] evaluated GPT-class models on procurement tasks and found them useful but prone to hallucination. Friedman et al. [5] took a different angle, showing that LLM suggestions can reduce cognitive biases in purchasing decisions. We draw on both findings:

Vyshakh Vijayan is with the Department of Computer Science and Business Systems, VIT-AP University, Amaravati, Andhra Pradesh, India. Email: vijayan.22bcb7055@vitapstudent.ac.in

StockSense uses Groq Llama 3.1 8B for reorder reasoning, but wraps it with a rule-based fallback so that a malformed API response never breaks the pipeline.

Workflow automation has quietly become central to small-business IT. Amir and Atif [6] benchmarked n8n against manual processes and found it cut implementation time significantly. Venkiteela [10] positioned n8n as a viable alternative to Zapier for enterprise integration. Our choice of n8n over a custom backend is driven by these findings—it lets one developer wire together eight workflows without writing server-side code.

On the supplier-evaluation side, Weighted Sum Model (WSM) approaches are standard in operations research. StockSense applies WSM with three criteria—reliability, price competitiveness, and delivery speed—weights that were chosen after reviewing common small-retailer priorities.

Finally, hardware-centric approaches deserve mention. Manoharan et al. [8] combined IoT sensors with ML for automated stock counting, and Poongothai et al. [9] proposed smart-retail forecasting. Both require RFID readers or IoT gateways, which adds Rs. 10,000–50,000 in upfront cost. For a kirana store, that is a non-starter. Shenoy and Mal [7] specifically studied Indian MSME inventory needs and called for "simple yet effective" tools—an ethos StockSense tries to follow by keeping the entire stack software-only.

III. Motivation and Problem Specification

There is a wide gulf between what the research community proposes and what a kirana store owner can actually use. Most published inventory models assume clean historical data, stable internet, and a dedicated IT person—none of which exist in a typical neighborhood shop.

A. Socio-Economic Motivation

Kirana stores account for roughly 80% of India's grocery retail, yet almost none of them use any digital tool beyond a UPI payment app. A single stockout—say, running out of Aashirvaad Atta during a festival week—can send a loyal customer to the competitor across the street. On the flip side, over-ordering perishable goods like bread or milk leads to waste. Both mistakes directly hit the owner's pocket. StockSense AI was motivated by this everyday pain point: can we give a small shopkeeper the kind of stock intelligence that a Reliance Fresh or DMart gets from its ERP, but without the cost?

B. Gap Analysis and Technical Advantage

Two specific gaps stood out during our literature review and field observations:

1) **Financial Barrier**: "Smart" inventory systems typically need RFID hardware or carry SaaS subscriptions costing Rs. 500–5,000 per month. StockSense keeps costs under Rs. 500/month by stitching together free-tier services—Supabase for the database, Groq for LLM inference, and a locally hosted n8n instance for automation. Enterprise tools like Zoho Inventory cost Rs. 4,000+ for comparable features.
2) **Intelligence Gap**: Apps like Vyapar and Khatabook are popular but reactive—they tell the owner what *was* sold, not what *will* run out. By plugging in an 8B-parameter language model, StockSense brings reasoning-based reorder logic to a segment that has never had it.

C. Problem Statement

The goal is straightforward: build a pipeline that takes raw inventory numbers, figures out which products are at risk, decides how much to reorder and from which supplier, and delivers the recommendation to the owner's WhatsApp—all without the owner lifting a finger.

IV. Proposed System

A. System Architecture

The system is organized into four layers, each handling a distinct responsibility. At the top sits the **Frontend Layer**—a React 19 single-page application styled with Tailwind CSS v4 and bundled by Vite 8. Below that, the **Workflow Engine Layer** runs on n8n and contains all eight business-logic pipelines, each triggered either by a webhook call from the frontend or by a cron schedule. The **Database Layer** is Supabase-hosted PostgreSQL; it stores all tables and pushes real-time change events to the frontend via WebSockets, so the dashboard updates without polling. Finally, the **AI and External Services Layer** wraps the Groq Llama 3.1 8B endpoint for generating reorder suggestions and the Twilio API for WhatsApp delivery.

Communication between layers follows two paths. The frontend talks to n8n through REST webhooks and reads from Supabase directly using its JavaScript client. n8n, in turn, reads and writes to Supabase via HTTP and calls Groq and Twilio through their respective REST APIs.

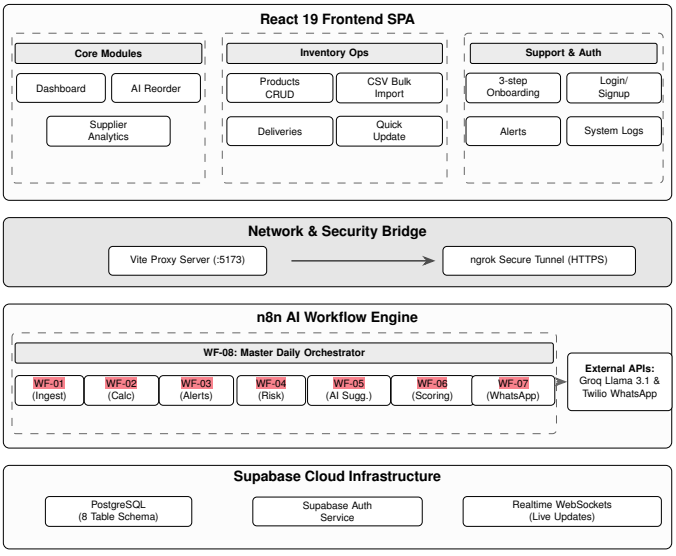

Fig. 1: StockSense AI System Architecture and Workflow Pipeline.

B. Mathematical Models

The system implements several mathematical models for inventory analysis and supplier evaluation:

Stockout Prediction Model (WF-02): The stockout date calculator computes the expected number of days until a product runs out of stock based on current inventory and sales velocity:

$$d_{\text{stockout}} = \left\lfloor \frac{S_{\text{current}}}{V_{\text{daily}}} \right\rfloor \tag{1}$$

$$\text{date}_{\text{stockout}} = \text{date}_{\text{today}} + d_{\text{stockout}} \tag{2}$$

where S_{current} is the current stock level and V_{daily} is the average daily sales velocity. If $V_{\text{daily}} = 0$, d_{stockout} is set to 999 as a safe default. If $S_{\text{current}} = 0$, $d_{\text{stockout}} = 0$ indicating immediate stockout.

Stock Risk Classification (WF-04): Products are classified into risk tiers based on d_{stockout}:

$$\text{risk} = \begin{cases} \text{HIGH} & \text{if } d_{\text{stockout}} \leq 3 \\ \text{MEDIUM} & \text{if } 3 < d_{\text{stockout}} \leq 7 \\ \text{LOW} & \text{if } d_{\text{stockout}} > 7 \\ \text{UNKNOWN} & \text{if no stockout data available} \end{cases} \tag{3}$$

Supplier Scoring using Weighted Sum Model (WF-06): The supplier scoring system uses a multi-criteria Weighted Sum Model to compute a composite score for each supplier:

$$C_s = (R \times 0.40 + P \times 0.35 + D \times 0.25) \times 100 \tag{4}$$

where R is the reliability score (0–1 scale), P is the price score (0–1 scale), and D is the delivery speed score (min-max normalized: fastest supplier = 1.0, slowest = 0.0). Grade assignment follows: A if $C_s \geq 80$, B if $65 \leq C_s < 80$, C if $50 \leq C_s < 65$, D if $C_s < 50$.

Inventory Health Score (Dashboard): The dashboard calculates an overall inventory health score:

$$H = \min\left(100, \max\left(0, \left\lfloor \frac{\sum(n_i \cdot w_i)}{N} \right\rfloor\right)\right) \tag{5}$$

where the numerator is $n_L \cdot 100 + n_M \cdot 70 + n_H \cdot 30 + n_O \cdot 5$, n_L, n_M, n_H, and n_O are counts of LOW, MEDIUM, HIGH risk, and out-of-stock products respectively, and N is the total number of products.

V. Implementation

A. Tools and Technologies

The technology choices were driven by two constraints: keep costs near zero and avoid anything that requires a dedicated backend developer. The frontend uses **React 19** with **Vite 8** for fast builds and **Tailwind CSS v4** for styling. On the backend side, **Supabase** provides a managed PostgreSQL database with built-in auth and real-time WebSocket subscriptions—no separate server needed. All business logic lives inside **n8n**, a visual workflow engine that we self-host locally during development. For AI, the system calls the **Groq API** which hosts Llama 3.1 8B and returns inference results in under two seconds. **Twilio** handles the last mile: formatting a stock-alert message and delivering it over WhatsApp.

B. Dataset

We did not use a publicly available dataset. Instead, we synthesized a small catalog that mirrors a real kirana store: six fast-moving products (rice, atta, cooking oil, soap, biscuits, and dal) with realistic daily sales rates, reorder thresholds, and opening stock levels. Three supplier profiles were created with varying reliability and delivery-speed scores to exercise the WSM scoring logic. This controlled setup lets us test edge cases—for instance, what happens when a product has zero daily sales, or when stock is already at zero—without relying on noisy real-world data.

C. Database Schema

The Supabase PostgreSQL database contains eight tables as described in Table I.

TABLE I: Database Tables in StockSense AI

Table	Purpose
Store Profiles	Multi-tenancy-ready store configuration
Products	Inventory items with stock, pricing, risk
Suppliers	Vendor directory with scoring and grades
Stock Alerts	Low stock alerts (stock $\leq$ threshold)
Reorder Suggestions	AI-generated reorder recommendations
Stock Transactions	Audit log of all stock changes
Daily Snapshots	Historical daily health metrics
System Logs	Workflow execution logs

All tables use UUID primary keys generated via `gen_random_uuid()`. The Products, Suppliers, Stock Alerts, and Reorder Suggestions tables include `store_id` foreign keys for multi-tenancy-ready schema design. Indexes are created on frequently queried columns including `product_id`, `risk_level`, `alert_status`, and `snapshot_date`. Supabase Realtime subscriptions are enabled on Products, Stock Alerts, Reorder Suggestions, Daily Snapshots, and Suppliers tables for live frontend updates.

D. Workflow Pipeline

Eight n8n workflows implement the inventory intelligence pipeline.

WF-01: Product and Sales Data Ingestion exposes a POST webhook at `/product-sales-ingest` that receives product data from the frontend Add Product modal. The workflow validates and upserts products into the *Products* table.

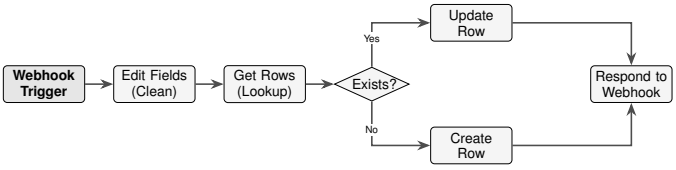

Fig. 2: WF-01: Data Ingestion Pipeline.

WF-02: Stockout Date Calculator fetches all products, applies the stockout prediction formula from (1), and updates each product record with `days_to_stockout` and `estimated_stockout_date`.

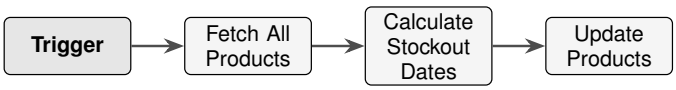

Fig. 3: WF-02: Stockout Prediction Pipeline.

WF-03: Inventory Processing & Stock Alerts identifies products where `current_stock` $\leq$ `reorder_threshold`, clears existing alerts, and generates fresh *Stock Alert* records.

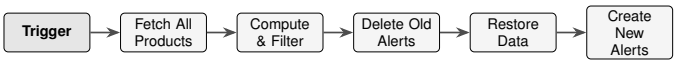

Fig. 4: WF-03: Inventory Processing & Alerts Pipeline.

WF-04: Stock Risk Classification classifies all products into risk tiers using the formula in (3) based on `days_to_stockout` values from WF-02.

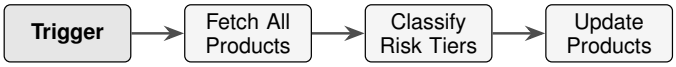

Fig. 5: WF-04: Risk Classification Pipeline.

WF-05: AI Reorder Intelligence fetches at-risk products and all suppliers, then invokes Groq Llama 3.1 8B with a structured prompt to generate reorder recommendations. The LLM output is parsed and stored as *Reorder Suggestions*.

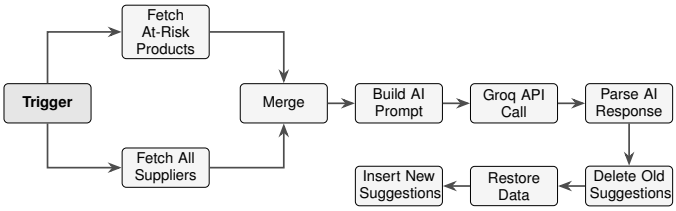

Fig. 6: WF-05: AI Reorder Intelligence Pipeline.

WF-06: Supplier Scoring evaluates all suppliers using the WSM formula from (4), computes composite scores, and assigns letter grades based on performance metrics.

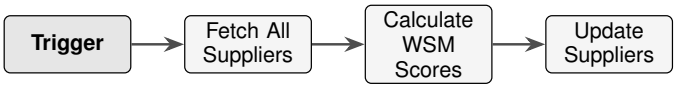

Fig. 7: WF-06: Supplier Scoring Pipeline.

WF-07: WhatsApp Alert Sender queries active stock alerts, filters by 24-hour anti-spam cooldown, builds a formatted message, and delivers it via Twilio WhatsApp API.

Fig. 8: WF-07: WhatsApp Alert Pipeline.

WF-08: Daily Orchestrator is the master workflow executing at 8:00 AM IST. It simulates daily stock depletion, sequentially executes WF-02 through WF-07 as sub-workflows, computes and saves the daily health snapshot, and logs execution to *System Logs*.

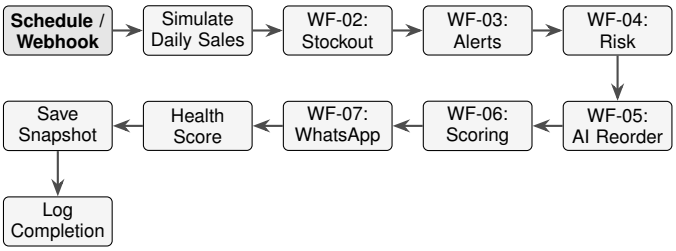

Fig. 9: WF-08: Master Daily Orchestrator Pipeline.

E. LLM Prompt Engineering

WF-05 constructs a structured prompt submitted to the Groq Llama 3.1 8B inference endpoint. The prompt follows a zero-shot instruction format comprising three components: (1) a role definition establishing the model as an inventory manager for an Indian kirana store; (2) structured context blocks listing at-risk products with fields `product_id`, `category`, `current_stock`, `avg_daily_sales`, `days_to_stockout`, and `risk_level`; and (3) available supplier profiles with `supplier_id`, `supplier_grade`, `delivery_time_days`, and `supplies_categories`.

The model is instructed to return a JSON array only, with no markdown fencing or natural language preamble. Each element contains `product_id`, `supplier_id`, `suggested_quantity` (computed as $30 \times v$ where v is average daily sales), and a `reason` field providing a single-sentence justification. Category matching between products and supplier inventories is enforced as case-insensitive. A rule-based fallback activates if the API response cannot be parsed as valid JSON, ensuring the system remains operational during inference failures.

F. Frontend Application

The React 19 frontend provides 12 routes as described in Table II.

TABLE II: Frontend Routes and Functions

Route	Function
/	Landing page
/login, /signup	Supabase Auth entry points
/reset-password	Password reset flow
/onboarding	3-step store setup wizard
/dashboard	Health score, KPIs, critical products, trend chart
/products	Product CRUD with CSV bulk import
/quick-update	Batch stock adjustments
/alerts	Stock alert feed with dismiss
/reorder	AI reorder suggestions with approve/dismiss
/suppliers	Supplier table with grade badges
/logs	System log viewer
/settings	Store profile and workflow triggers

The Dashboard implements a real-time pipeline overlay: when the user triggers WF-08, an animated modal cycles

through realistic step messages while a Supabase Realtime subscription on System Logs listens for the pipeline completion insert. The moment n8n finishes, the overlay closes and the dashboard refreshes automatically without polling.

VI. Results and Performance

The system was tested end-to-end with six products and three suppliers. Table III lists the key numbers.

TABLE III: System Performance Metrics

Metric	Value
Number of Workflows	8
Number of Database Tables	8
Number of Frontend Routes	12
Pipeline Execution Time	∼13 seconds
WhatsApp Message Delivery	<5 seconds
Infrastructure Cost	Rs. 0 (prototype); <Rs. 500/mo (production)

Table IV puts StockSense side by side with three apps that Indian retailers commonly use today. The main takeaway is that none of the alternatives offer AI-based reorder logic or automated WhatsApp alerts at a comparable price point.

TABLE IV: Comparison with Existing Inventory Solutions

Feature	StockSense AI	Vyapar	Zoho Inventory	Khatabook
AI Reorder Suggestions	Yes	No	Limited	No
Stockout Prediction	Automated	Manual	Basic	No
Supplier Scoring	WSM-based	No	Basic	No
WhatsApp Alerts	Automated	No	No	No
Monthly Cost	Rs. 0–500	Rs. 0–599	Rs. 4,000+	Rs. 0
Setup Complexity	Low	Medium	High	Low
Prediction Method	Rule-based	No	Basic	No
Real-time Dashboard	Yes	Limited	Yes	No

Health Score Validation

To make sure the health-score formula from (5) behaves sensibly, we ran it against four inventory states that a kirana store might realistically encounter. Table V shows the results.

TABLE V: Health Score Across Inventory States

Scenario	n_L	n_M	n_H	n_O	N	H
All well-stocked	6	0	0	0	6	100
Demo state	1	2	3	0	6	55
Partial stockout	0	1	2	3	6	18
Complete stockout	0	0	0	6	6	5

The numbers make intuitive sense. A store with everything in stock gets a perfect 100; the demo dataset (three high-risk items) lands at 55; and a store where three products are completely out scores just 18, which would trigger an urgent alert. We deliberately set the floor at 5 instead of 0 so that the dashboard gauge never shows a completely empty reading, which could confuse the user into thinking the system is broken.

On the cost side, the prototype runs entirely on free tiers: Supabase gives 500 MB of PostgreSQL storage, Groq offers rate-limited LLM inference at no charge, and n8n runs on the developer's laptop. Twilio WhatsApp works in sandbox mode during testing. Moving to production would require hosting n8n on a cloud platform like Railway or Render (roughly Rs. 300–500 per month) and paying Twilio's per-message fees. Even so, the total monthly bill for one store stays well below Rs. 500, which is an order of magnitude cheaper than Zoho Inventory.

The daily pipeline ran without errors across multiple test cycles. Each morning at 8:00 AM, n8n's cron trigger kicks off the master workflow, and all eight sub-pipelines execute in sequence. The only caveat is that this schedule only works while the n8n instance is actually running and reachable.

WhatsApp turned out to be the right delivery channel. Indian shopkeepers check WhatsApp far more often than email, and the built-in 24-hour cooldown we added prevents the same alert from being sent repeatedly. During testing, messages arrived consistently within five seconds of the pipeline finishing.

The health score proved useful as a single-glance metric. Rather than scanning a table of six products, the owner sees one number on the dashboard and immediately knows whether to worry.

The AI reorder suggestions were the most interesting part to evaluate. Groq Llama 3.1 8B takes the product list, the supplier profiles, and the risk levels, and returns a JSON array with quantity suggestions and a one-line justification for each. The reasoning is not always perfect—occasionally the model suggests ordering from a supplier who does not carry that category—but the rule-based fallback catches these cases and defaults to a $30 \times v$ formula.

VII. Conclusion

StockSense AI set out to answer a simple question: can a small Indian shopkeeper get useful inventory intelligence without paying enterprise prices? Based on our prototype, the answer is a cautious yes. The system automates the entire loop—from tracking stock levels to sending a WhatsApp message that says "order 30 packets of Aashirvaad Atta from Supplier A"—and it does so for under Rs. 500 a month in production.

That said, several limitations remain. The stockout prediction is rule-based, not learned from data, so it cannot account for demand surges during festivals. The LLM occasionally produces malformed output, which the fallback catches but does not improve upon. And the 8:00 AM schedule only works if the n8n instance is online, which requires cloud hosting that a non-technical owner cannot manage alone.

VIII. Future Scope

The current version is a working baseline. Below are the extensions we consider most impactful.

Barcode Scanning: Right now, products are added by typing in details manually. Hooking into the phone camera for barcode or QR lookups would cut onboarding time significantly. The existing `product_id` field already accepts barcode strings, so no schema change is needed.

Voice Input in Regional Languages: Many kirana owners are more comfortable speaking than typing. A voice interface accepting Telugu, Hindi, or Marathi commands like "5 packet Aashirvaad Atta aaya" would make stock updates much faster. The Web Speech API is browser-native and would not add infrastructure cost.

Two-way WhatsApp: Currently, WhatsApp is one-directional—the system sends alerts, but the owner cannot reply to approve a reorder. Wiring up a Twilio inbound webhook would let owners respond with "yes" or "no" directly from the chat, removing the need to open the dashboard.

Seasonal Adjustments: The Products table already has `is_seasonal` and `peak_season_months` columns that are not yet active. Turning them on would let the system multiply `avg_daily_sales` by a seasonal factor before Diwali or Holi, improving stockout predictions during peak demand.

Multi-store Support: The schema includes a `store_id` foreign key in every table, so the database is already multi-tenancy-ready. Adding Supabase Row Level Security policies scoped to `store_id` would let multiple stores share one deployment without seeing each other's data.

Purchase Orders: Once a reorder suggestion is approved, the next logical step is generating a formal purchase order with a UPI payment link. This would close the procurement loop entirely within the app.

REFERENCES

[1] J. Wang, P. Li, L. Luo and H . Sun, "Design and Development of Intelligent Inventory System for Small and Micro Enterprise Warehousing," *IEEE Conf. on Computer, Big Data and AI*, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10550978/

[2] N. Kheawpeam and S. Sinthupinyo, "Demand Forecasting Using Machine Learning to Manage Product Inventory for Multi-channel Retailing Store," *IEEE Int. Conf. on Omni-layer Intelligent Systems*, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10189241/

[3] Jaymin V. Soni and Chetansinh Vaghela, "Retail Demand Forecasting in Inventory with External Influences: A Comparative Study of Machine Learning Models," *IEEE Trans. on Retail Tech.*, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/11232957/

[4] Mohammad Al Khaldy and Youcef Gheraibia, "Evaluation and Mitigation of the Limitations of Large Language Models in Business Decision-Making," *IEEE Access*, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/11013278/

[5] Natalie Friedman, Marcus Krug. and S. Joy Mountford, "Mediating Cognitive Biases in Business Decisions Using LLMs," *IEEE Trans. on AI*, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/11050471/

[6] Ahmed Raza Amir and Syed Muhammad Atif, "Evaluating Workflow Automation Efficiency Using n8n: A Small-Scale Business Case Study," *arXiv preprint arXiv:2602.01311*, Feb. 2025. [Online]. Available: https://arxiv.org/abs/2602.01311

[7] Dinesh Shenoy and Hoshiar Mal, "A Multi-item Inventory Model for Small Business—A Perspective from India," *Int. J. of Inventory Research*, vol. 5, no. 3, pp. 188–209, 2019. [Online]. Available: https://www.inderscienceonline.com/doi/abs/10.1504/IJIR.2019.098856

[8] Geetha Manoharan, Vipin Kumar, A Karthik, V Asha, Chinnem Rama Mohan, and Ginni Nijhawan, "IoT-Based Smart Inventory Management System Using Machine Learning Techniques," *IEEE Internet of Things Journal*, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10601903/

[9] K. Poongothai, Devika. G, Sweety Brisila. D, and Yogesh. S, "Smart Retail Using Machine Learning for Demand Forecasting and Inventory Optimization," *IEEE Trans. on Smart Business*, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10910874/

[10] Padmanabhan Venkiteela, "n8n: An Open-Source Workflow Automation Platform for Enterprise Integration and AI-Driven Orchestration," *Int. J. of Computer Applications*, vol. 187, no. 63, Dec. 2025. [Online]. Available: https://www.ijcaonline.org/archives/volume187/number63/